Transformer Model Architecture

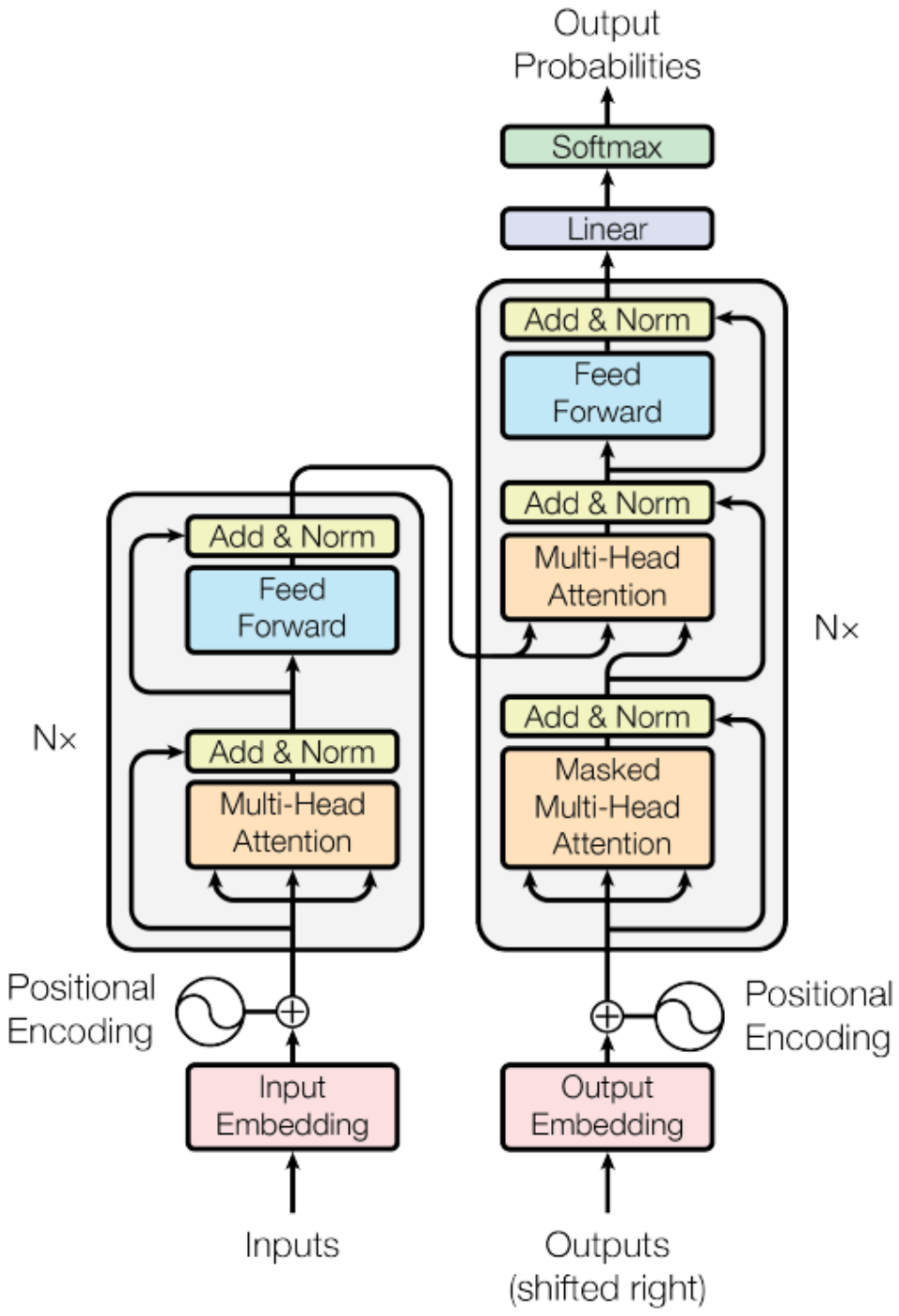
Natural Language Processing (NLP)

Sanjaya Edirisinghe
sanjayae@bu.edu

Transformer Model Architecture

CONTENT :

1. Features of Transformers
2. Why Transformers ?
3. Self-Attention Mechanism
4. **QKV** Vectors
5. Self-Attention Mechanism steps
6. Multi-Head Attention
7. The Encoder-Decoder Structure
8. Positional Encoding
9. Feed Forward Network
10. Layer Normalization
11. Residual Connections



Features of Transformers

"Transformers represent a paradigm shift in processing sequential data for natural language understanding and generation, replacing traditional recurrent and convolutional layers with an innovative self-attention mechanism that enables efficient parallel processing."

Features :

- Self-Attention Mechanism
- Encoder-Decoder Architecture:
- QKV Vectors
- Layer Normalization
- Positional Encoding
- Feed-Forward Networks
- Residual Connections

Examples:

- BERT (Google)
- GPT (Open AI)
- ViT - Vision Transformer (Google)
- T5 - Text-to-Text Transfer Transformer (Google)

Why Transformers over RNN,LSTMs ?

Sequential Processing Bottleneck

RNNs and LSTMs process data sequentially, making them inherently slow and difficult to parallelize. Transformers use self-attention mechanisms to process input data in parallel, drastically improving computational efficiency and scalability.

Vanishing Gradient Problem

RNNs and LSTMs struggle with capturing long-range dependencies due to the vanishing gradient issue during backpropagation. Transformers overcome this by using self-attention, which directly connects all input tokens, allowing for the effective modelling of long-range dependencies.

Limited Context Window

LSTMs have a finite memory size, restricting their ability to capture relationships between distant tokens in a sequence. Transformers employ positional encoding and self-attention to model dependencies across the entire sequence, regardless of its length.

Gradient Flow Across Long Sequences

In RNNs, information has to flow through many time steps, which can cause gradient degradation over long sequences. Transformers avoid this issue by enabling direct token-to-token interaction through the self-attention mechanism.

Fixed-Length Input Dependency

RNNs and LSTMs often require fixed-length inputs or padding, which can introduce inefficiencies and noise. Transformers naturally handle variable-length inputs without padding through positional encoding and attention mechanisms, making them more flexible.

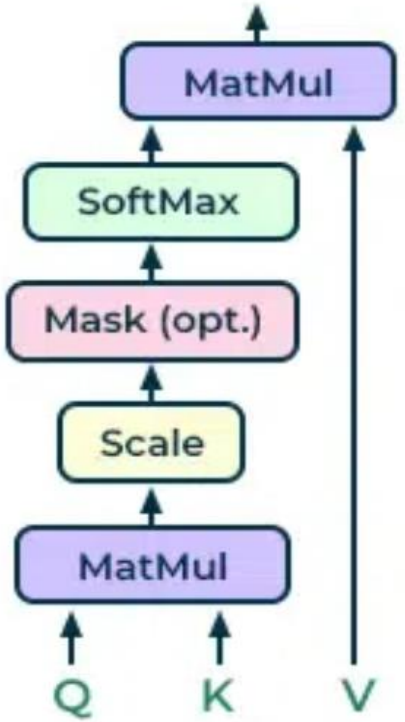
Self-Attention Mechanism

Self-attention computes a representation of a sequence where each element can "attend to" or focus on other elements in the sequence. It determines how much weight each element of the sequence should contribute to the representation of another element, effectively capturing contextual relationships.

Self-attention is crucial for understanding the context and meaning of words in sentences, enhancing the model's language processing capabilities.

$$\text{Self-Attention Output} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Steps:



Q(query)-**K**(key)-**V**(value) Vectors

Query (Q), **Key (K)**, and **Value (V)** vectors are central to the **self-attention mechanism** in transformers. These vectors are used to compute how much attention each element in a sequence (like a word or token) should pay to others.

Q

- What the current token (or element) **is looking for in the other tokens** (“Question”)
- *E.g. In the sentence "The cat chased the mouse," the query for "chased" might ask, "Who is chasing, and what is being chased?"*

K

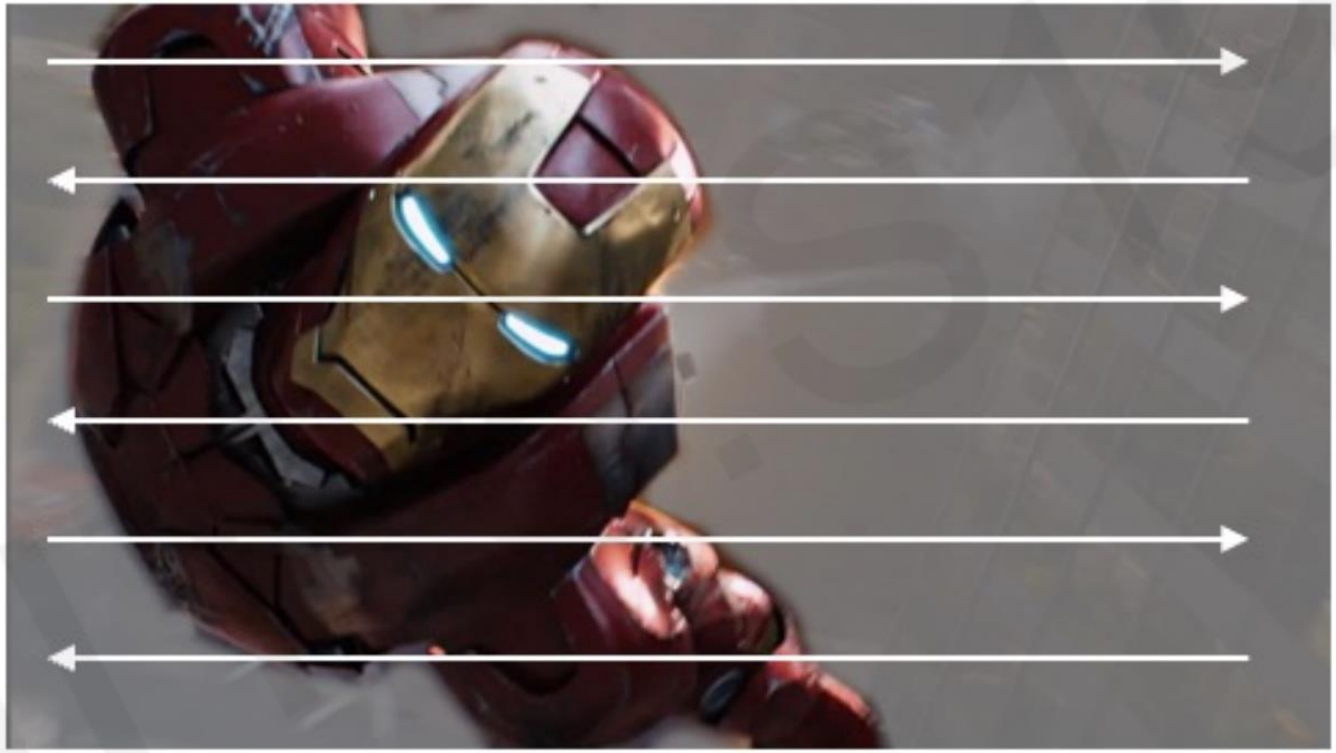
- **Information each token holds** that could be relevant to answering a query (“tag”)
- *E.g. The key for "cat" might encode the fact that it's a subject capable of performing actions like chasing*.

V

- This is the **actual data or content** of the token that will be used if the token is deemed relevant. (answer" to the query, weighed by importance (attention score).
- *E.g the value for "mouse" might include details about what is being chased*.

Intuition Behind Self-Attention

Attending to the most important parts of an input.



1. Identify which parts to attend to
2. Extract the features with high attention

Similar to a search problem!

Self-Attention Mechanism steps

1. **Input Representation**:
    - Input is a sequence of n vectors (e.g., words in a sentence) with dimensionality d, represented as $X = [x_1, x_2, \ldots, x_n]$, where $x_i \in \mathbb{R}^d$.

2. **Compute Query, Key, and Value Matrices**:
    - Each input vector is linearly transformed into three different vectors:
        - **Query** (Q): Represents the element looking for relevant information.
        - **Key** (K): Represents the element holding information.
        - **Value** (V): Represents the actual information carried by the element.
    - These are computed using learned weight matrices:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

where W^Q, W^K, W^V are learnable weight matrices.

Self-Attention Mechanism steps..ctd

3. **Calculate Attention Scores**:
   - The attention score is the dot product of the query and key vectors, scaled by the square root of the dimensionality d_k:

$$\text{Attention Score}(q_i, k_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

   This measures the relevance between the i-th query and j-th key.

4. **Apply Softmax**:
   - The scores are passed through a **softmax** function to convert them into probabilities, emphasizing the most relevant elements while downweighting less relevant ones:

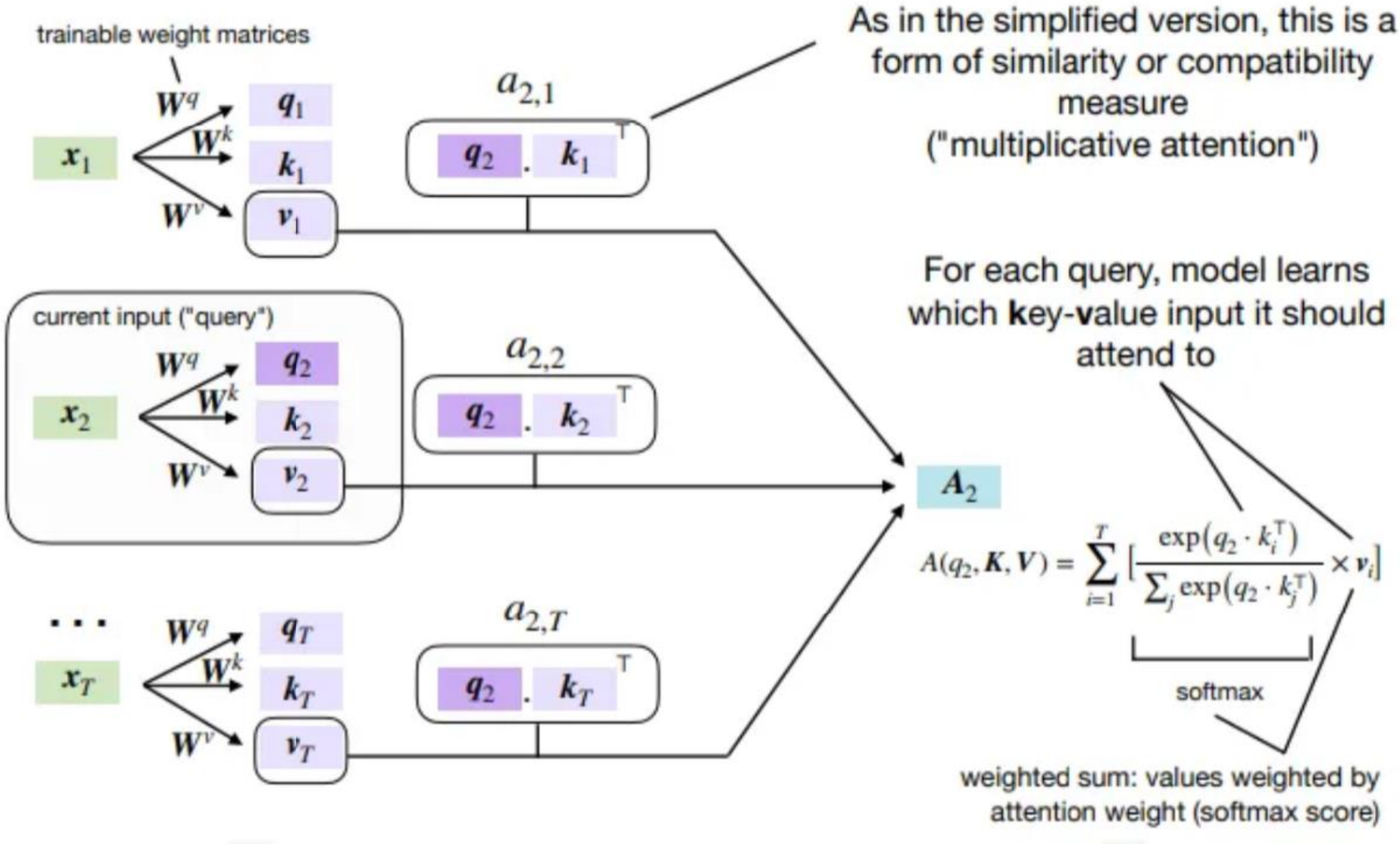
$$\text{Attention Weights} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

5. **Weight the Values**:
   - The attention weights are used to compute a weighted sum of the value vectors:

$$\text{Output} = \text{Attention Weights} \cdot V$$

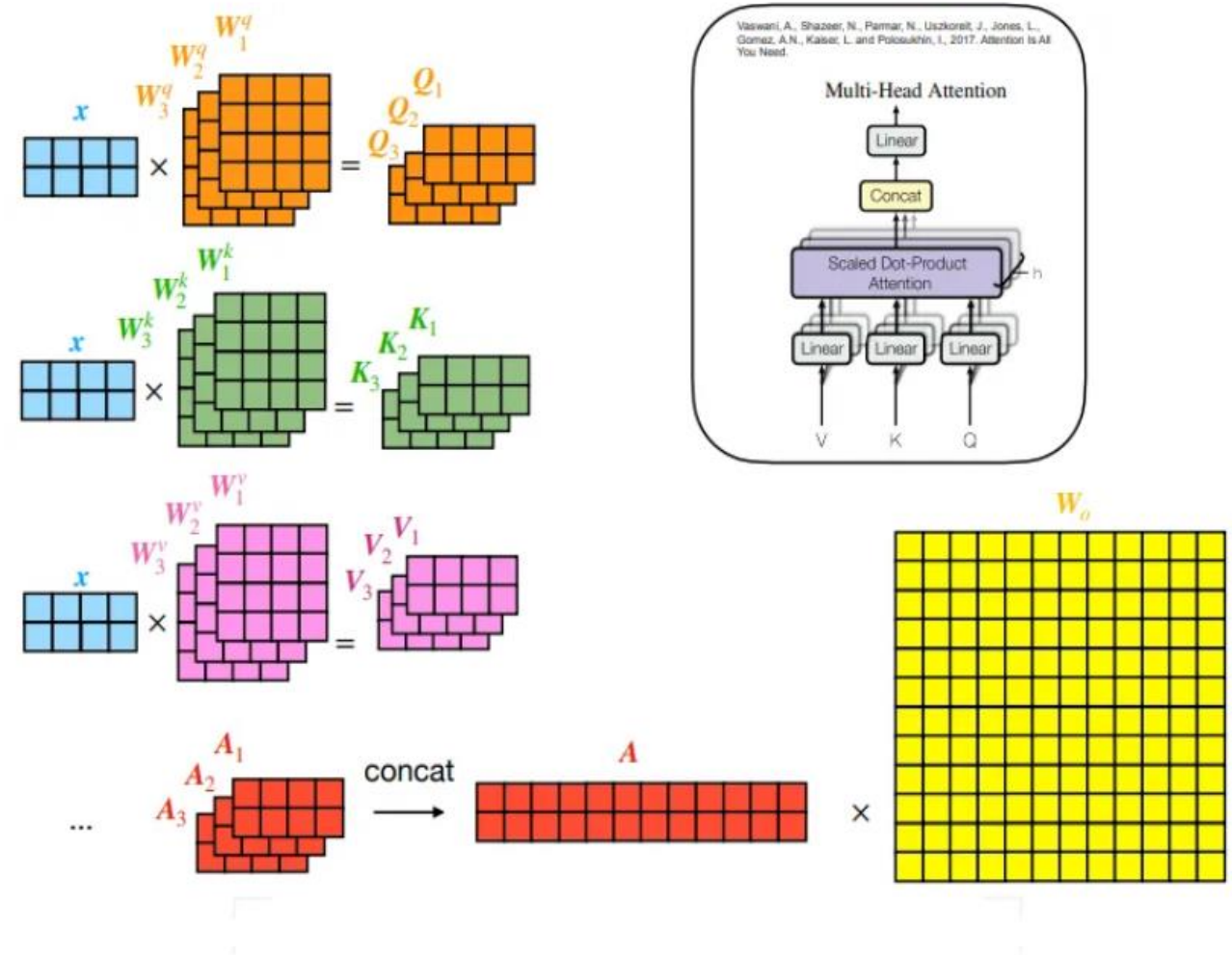
   This gives a contextualized representation for each element in the sequence.

Self-Attention Mechanism



Multi-Head Attention

- Extending the concept of self-attention,
- Help running several attention mechanisms (Muti-head attention) in parallel on both the encoder and decoder
- focus on different parts of the sentence for a given word in parallel
- enhances the model's ability to capture various linguistic features, such as syntax and semantics, from different perspectives



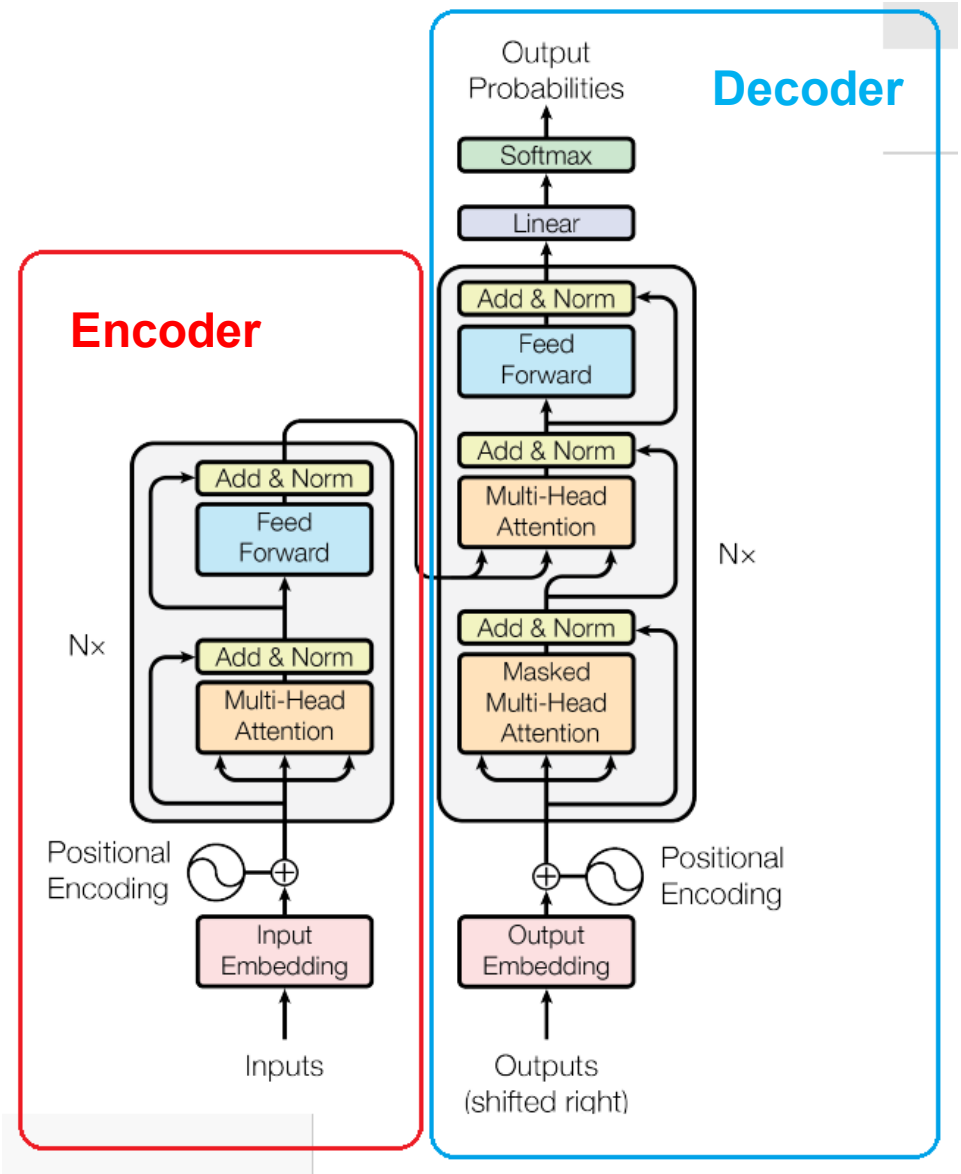
The Encoder-Decoder Structure

Encoder layer performs two primary functions:

- Self-attention (Process the input sequence and map it into a continuous representation that holds both the semantic and syntactic information of the input)
- Position-wise feed-forward neural networks

Decoder

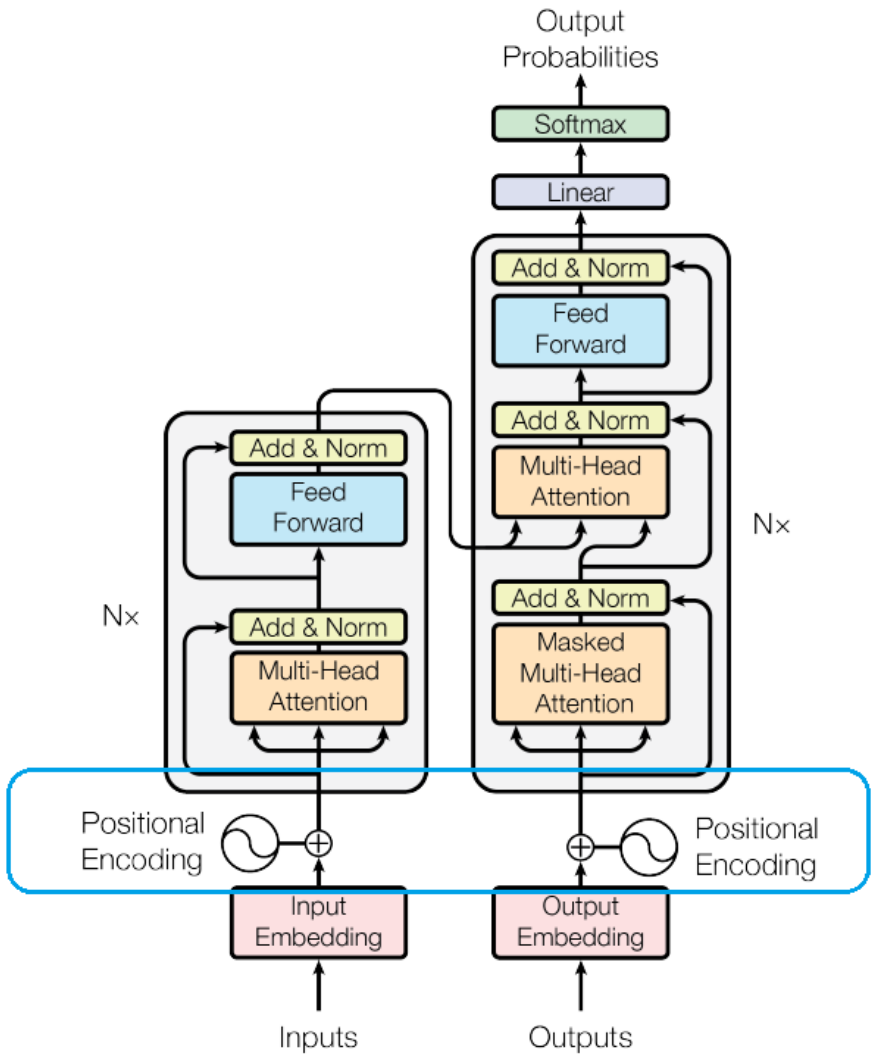
- Generate the output sequence from the encoded representation, one element at a time, with the assistance of the encoder's output and previously generated elements



Source: White paper "Attention Is All You Need" by Google 2017

Positional Encoding

- Transformers **does not** perform **data sequentially** like in recurrent models (e.g. RNN, CNN etc)
- Languages require to maintain some sequence in words (or tokens) for understanding the context
- Transformers use "**Positional encoding**" for the position of each word in the sequence to its representation.
- Positional encoding ensures that the model can consider the order of words, preserving the sequential nature of language.



Positional Encoding

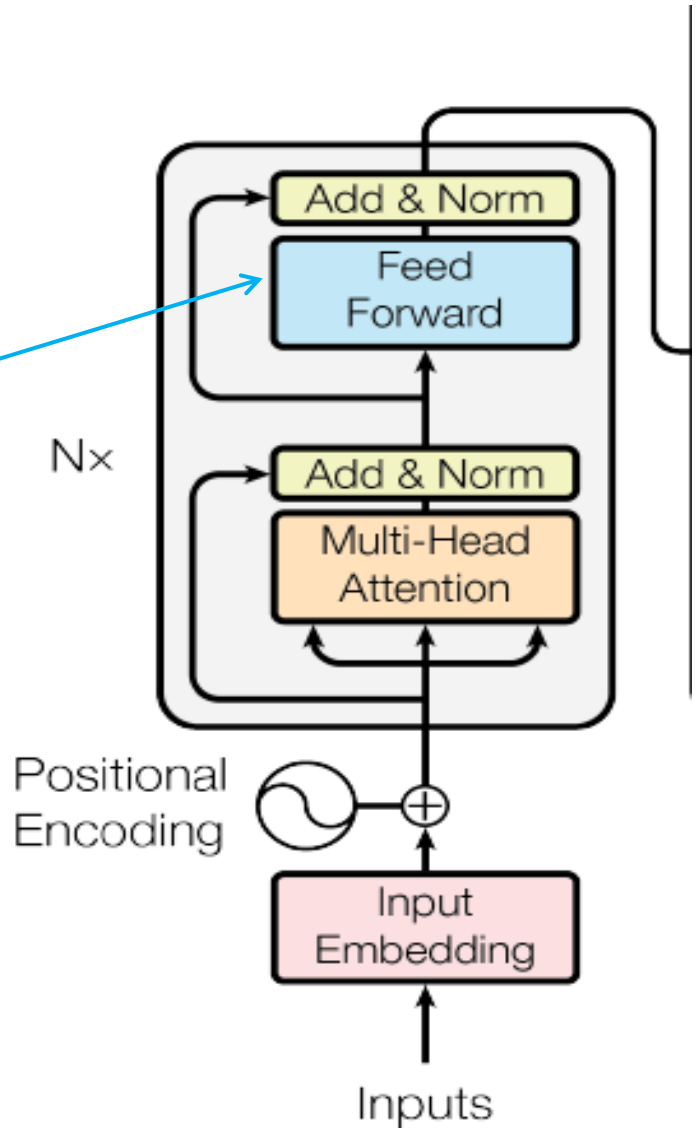
Source: White paper "Attention Is All You Need" by Google 2017

Feed Forward Network

Each layer within the encoder and decoder stacks incorporates a feed-forward neural network that processes **each position** of the sequence separately.

This **parallel architecture** allows for nuanced processing of sequences, with each layer adding a layer of understanding and refinement to the data representation

Feed Forward

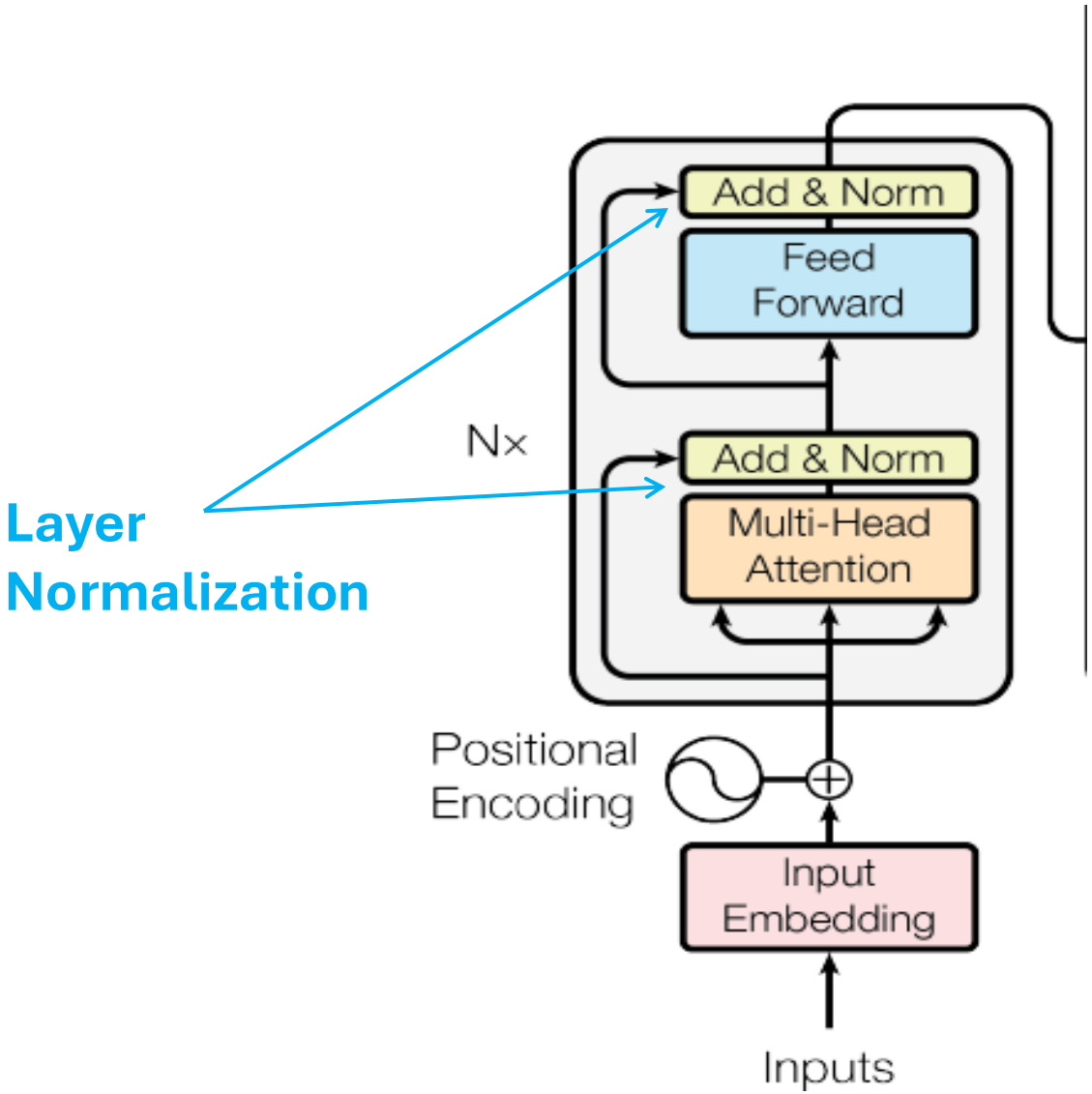


Source: White paper “Attention Is All You Need” by Google 2017

Layer Normalization

Stabilize the output of each sub-layer before it's passed on to the next layer

Ensures that the model trains more efficiently and effectively, reducing the training time significantly.

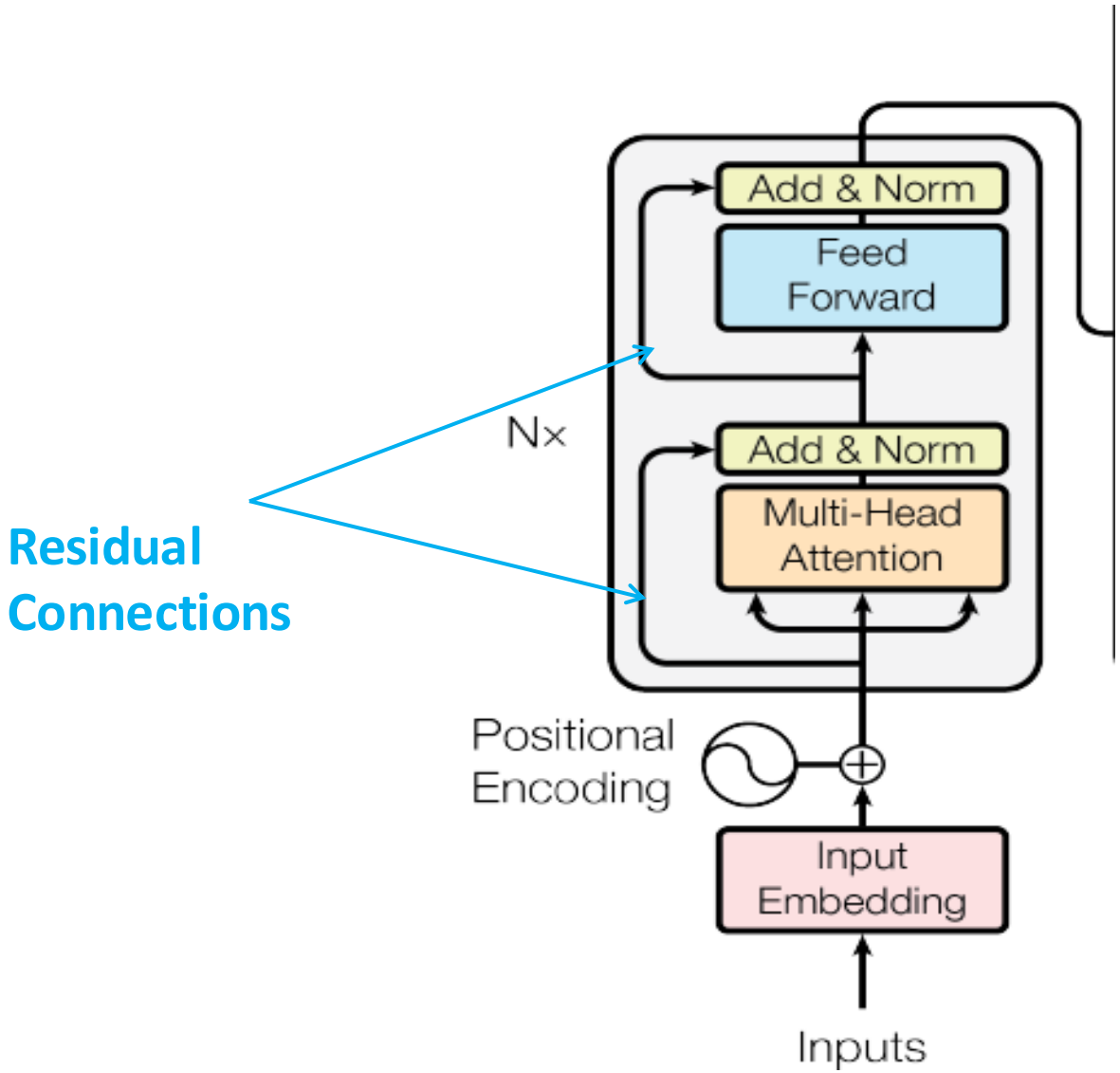


Source: White paper "Attention Is All You Need" by Google 2017

Residual Connections (Skip connections)

- Residual connections, (skip connections) allow the input of each sub-layer to be added to its output to mitigate the vanishing gradient problem in deep networks.

- Ensure that gradients can flow more freely through the network in deeper model architectures

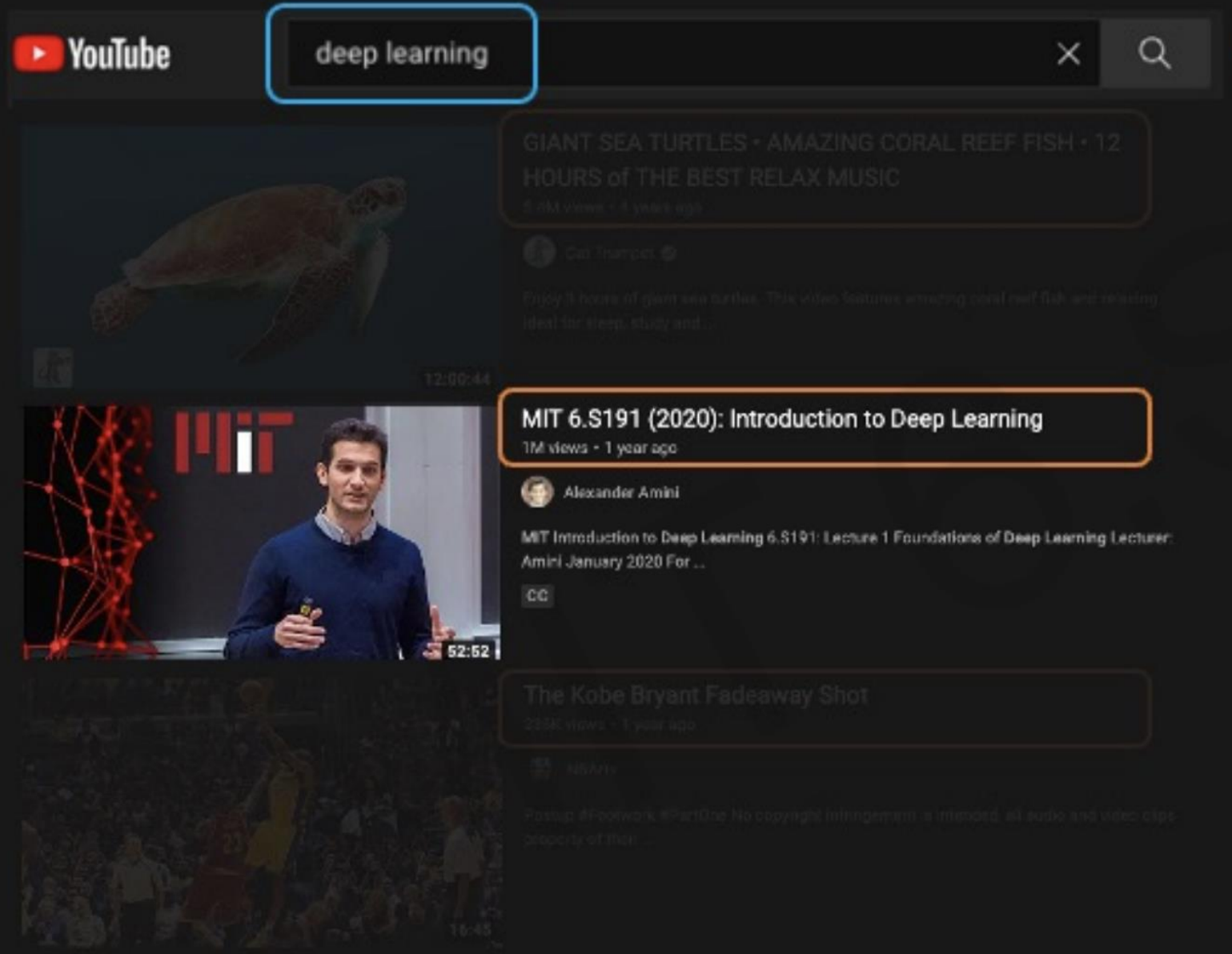


Source: White paper "Attention Is All You Need" by Google 2017

Attention Is All You Need

A Simple Example: Search

Understanding Attention with Search



Query (Q)

Key (K_1)

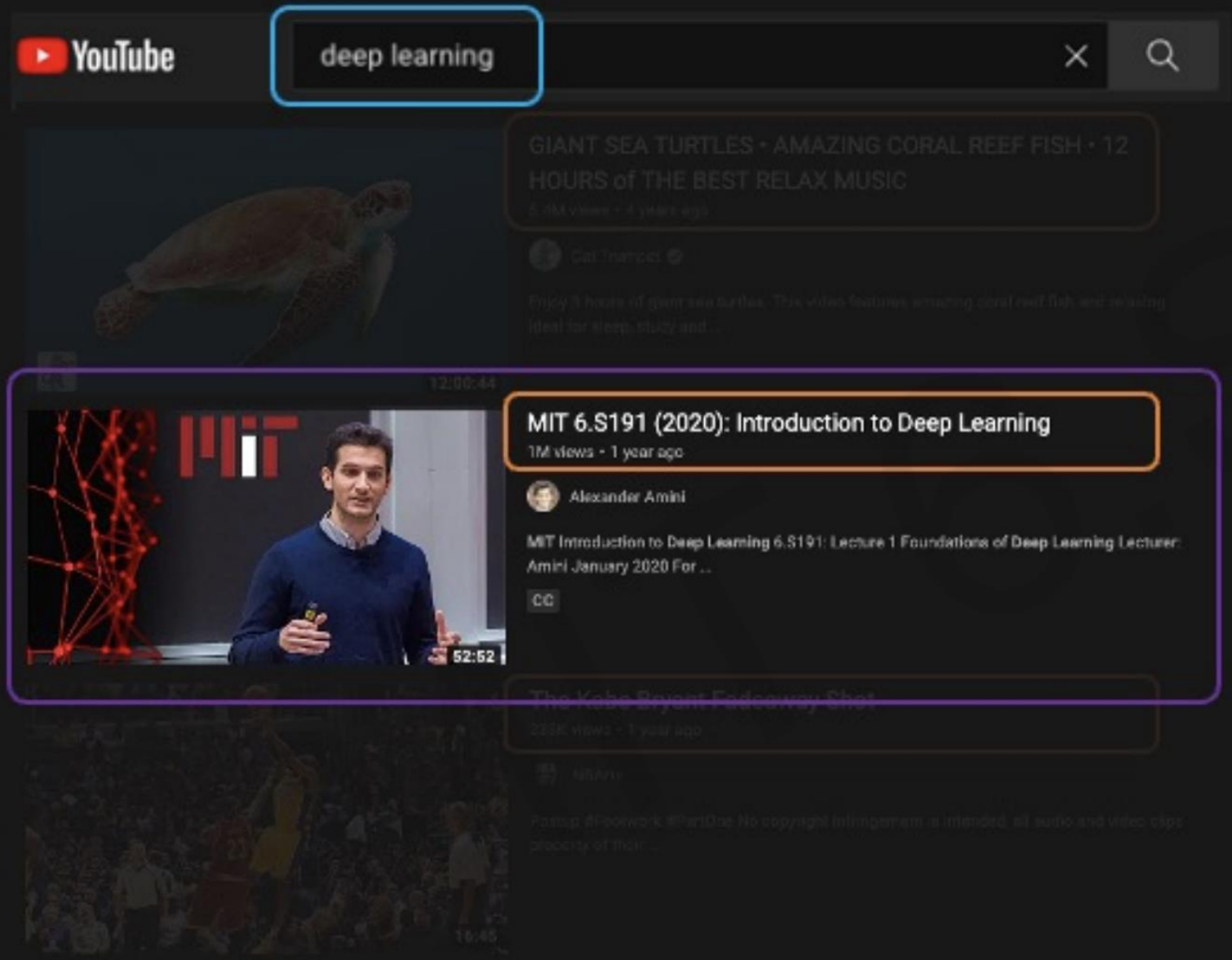
Key (K_2)

Key (K_3)

How similar is the key to the query?

1. **Compute attention mask:** how similar is each key to the desired que

Understanding Attention with Search



Query (Q)

Key (K_1)

Key (K_2)

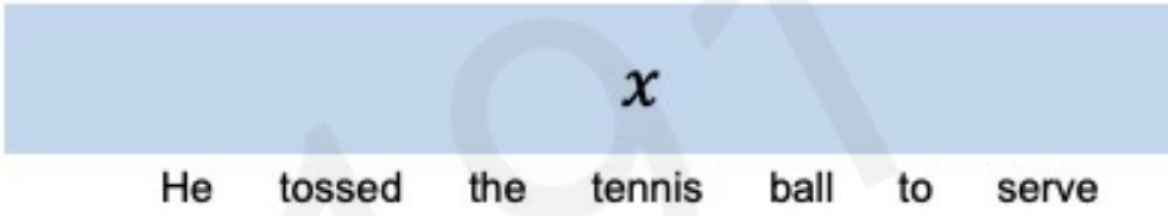
Value (V)

Key (K_3)

2. **Extract values based on attentio**
Return the values highest attention

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.



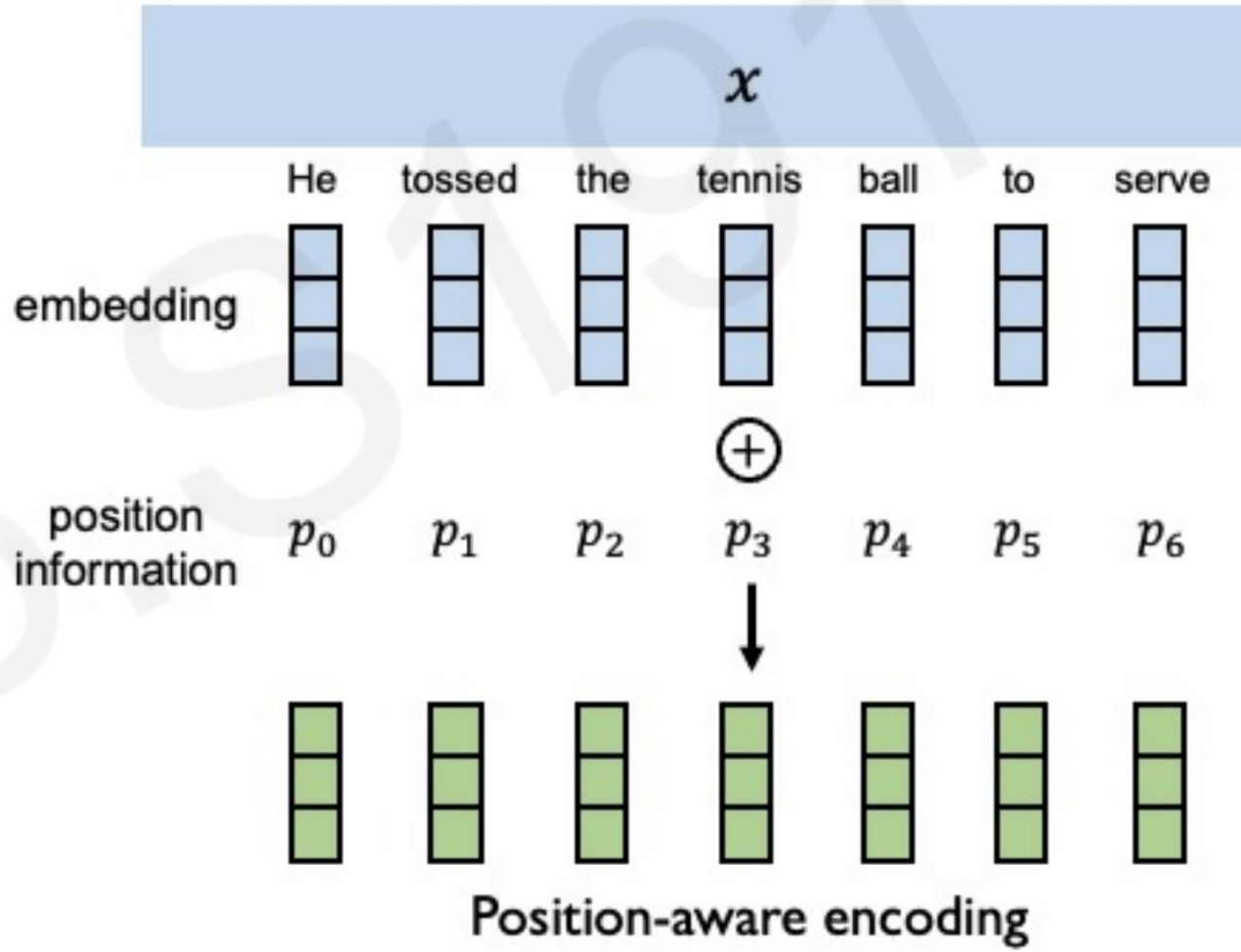
1. Encode **position** information
2. Extract query, key, value for search
3. Compute attention weighting
4. Extract features with high attention

Data is fed in all at once! Need to encode position information to understand order.

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract query, key, value for search
3. Compute attention weighting
4. Extract features with high attention

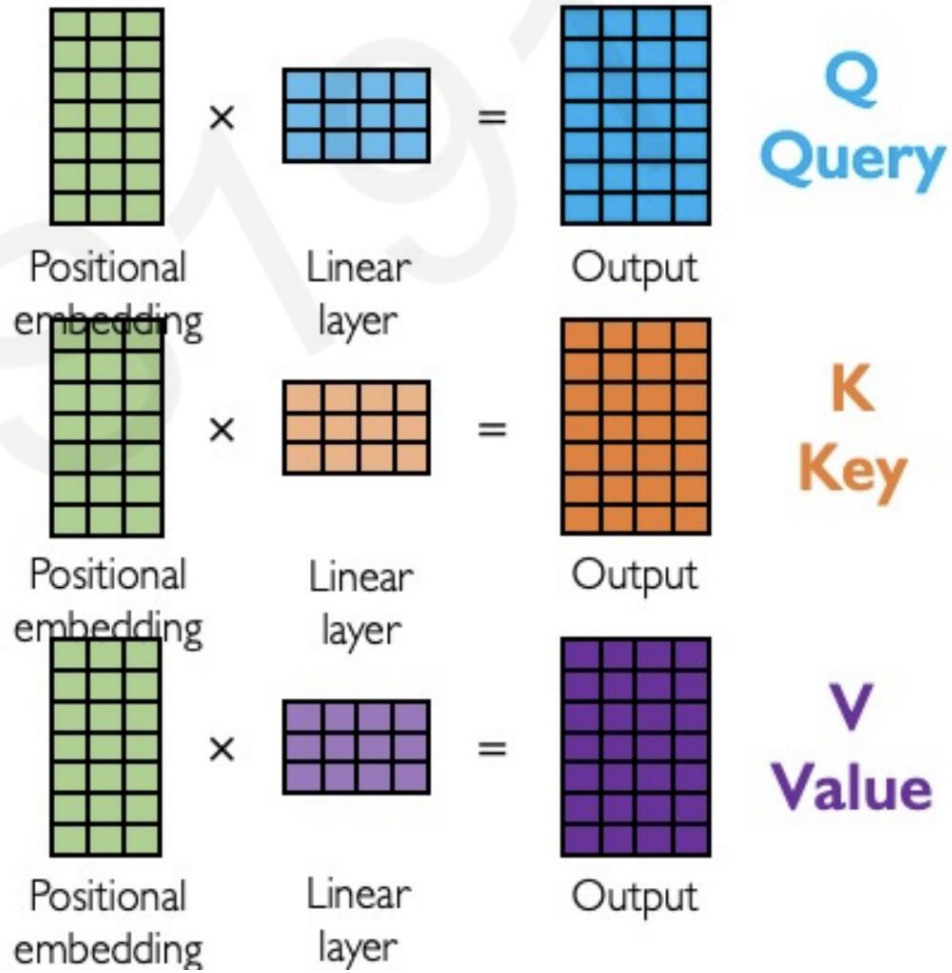


Data is fed in all at once! Need to encode position information to understand order.

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute attention weighting
4. Extract features with high attention



Recap : Why Transformers over RNN,LSTMs ?

Sequential Processing Bottleneck

RNNs and LSTMs process data sequentially, making them inherently slow and difficult to parallelize. Transformers use self-attention mechanisms to process input data in parallel, drastically improving computational efficiency and scalability.

Vanishing Gradient Problem

RNNs and LSTMs struggle with capturing long-range dependencies due to the vanishing gradient issue during backpropagation. Transformers overcome this by using self-attention, which directly connects all input tokens, allowing for the effective modelling of long-range dependencies.

Limited Context Window

LSTMs have a finite memory size, restricting their ability to capture relationships between distant tokens in a sequence. Transformers employ positional encoding and self-attention to model dependencies across the entire sequence, regardless of its length.

Gradient Flow Across Long Sequences

In RNNs, information has to flow through many time steps, which can cause gradient degradation over long sequences. Transformers avoid this issue by enabling direct token-to-token interaction through the self-attention mechanism.

Fixed-Length Input Dependency

RNNs and LSTMs often require fixed-length inputs or padding, which can introduce inefficiencies and noise. Transformers naturally handle variable-length inputs without padding through positional encoding and attention mechanisms, making them more flexible.

Recommended Resources

- https://www.youtube.com/watch?v=zxQyTK8quyY&t=2053s : Transformer Neural Networks, ChatGPT's foundation
- https://www.youtube.com/watch?v=kWLed8o5M2Y
- https://medium.com/@imad14205/deep-dive-into-the-transformer-architecture-pioneering-advances-in-nlp-and-large-language-model-b1f17d68d700 -- Encoder decoder architecture
- https://arxiv.org/abs/1706.03762 -- Attention is all you need, the paper by Ashish Wasvani